DEVDIRECTORY

Project flow, architecture & interface guide

A React + TypeScript developer discovery platform

This guide maps the main user journeys, describes the application architecture, and shows representative interface previews. The project includes developer discovery, profiles, jobs, interview practice, resume tools, workspace features, and an assistant.

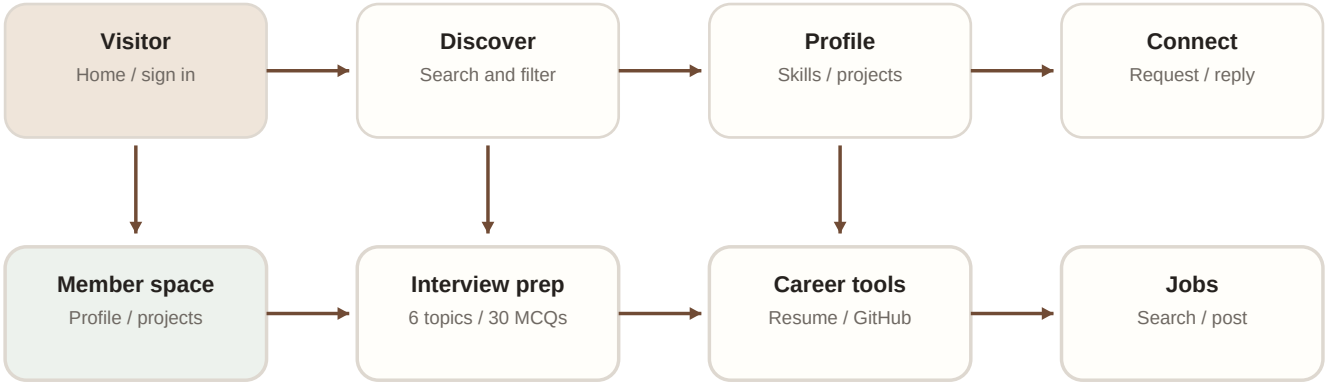
At a glance

Frontend	**React 18 / TypeScript / Vite / React Router**
UI	Responsive ivory / espresso visual system / Lucide icons
Data modes	Browser local demo mode or optional Supabase-connected mode
Interview prep	30 questions across DSA, React, Java, JavaScript, HTML, and CSS
Entry point	main.tsx -> App.tsx -> routed pages / features

Project folder: DevDirectory-React-Project. Start locally with **npm install** followed by **npm run dev**.

1. Product flow

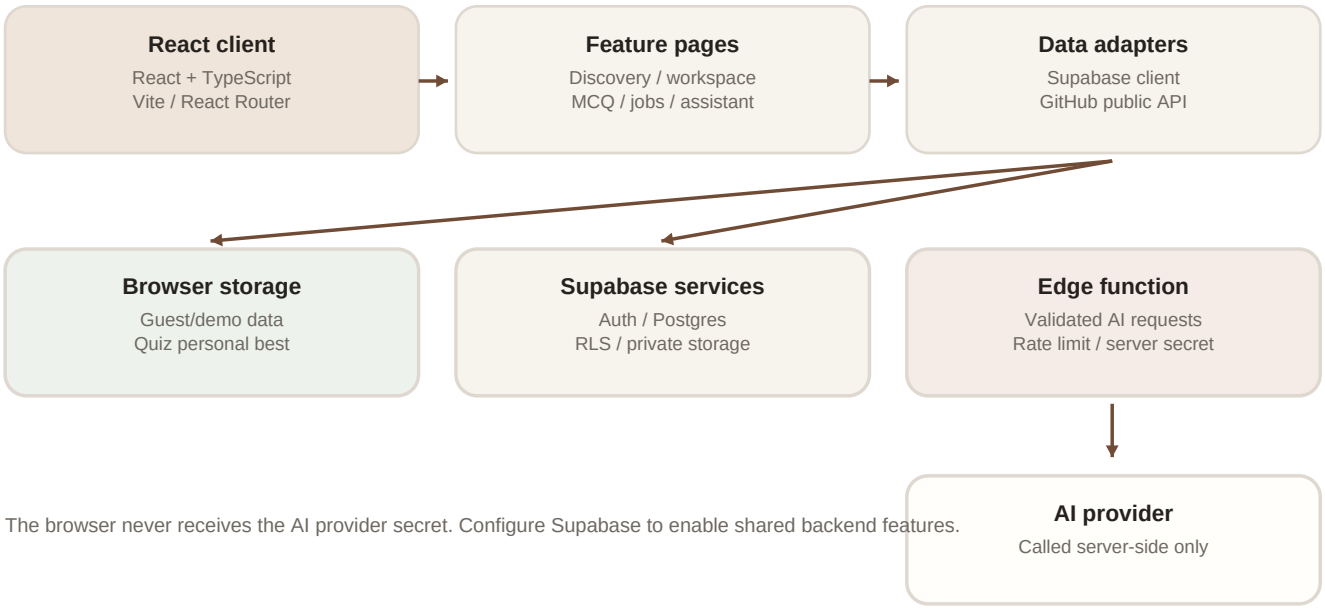
The app supports both discovery and career preparation. A visitor can browse public developer profiles, while signed-in members manage their own profile, projects, jobs, and workspace.



Main flow: discover developers and collaborate, or use career preparation and job tools.

2. Architecture

The React client handles routing, forms, validation, and the interface. When configured, Supabase provides authentication, shared records, row-level security, and private resume storage. The AI Edge Function keeps provider credentials on the server.



The browser never receives the AI provider secret. Configure Supabase to enable shared backend features.

Key source files

App.tsx - Routes, account state, navigation, and primary page composition.

Practice.tsx / practice.css - Interview MCQ bank, topic selection, answers, explanations, score, and personal best.

Features.tsx / features.css - Jobs, admin, career tools, and assistant features.

Workspace.tsx / workspace.css - Project board, tasks, goals, and activity.

supabase-client.ts - Optional frontend Supabase client configuration.

database-schema.sql - Database tables, policies, and private storage setup.

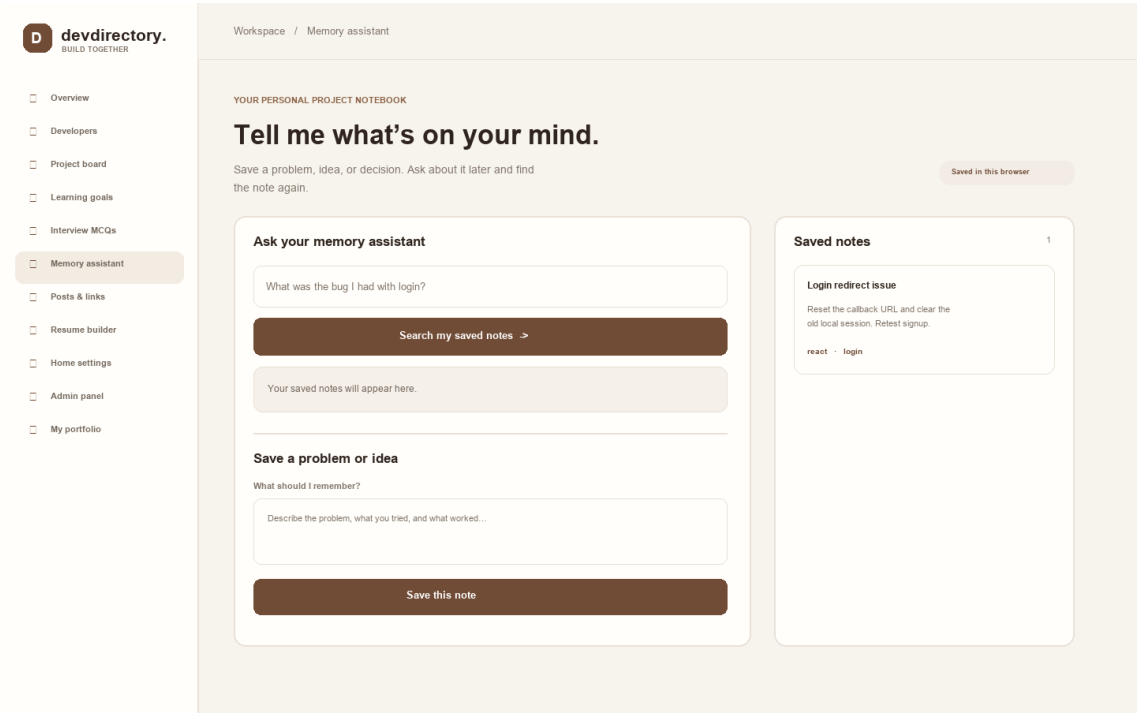
developer-assistant-function.ts - Server-side assistant function source.

Demo mode vs connected mode

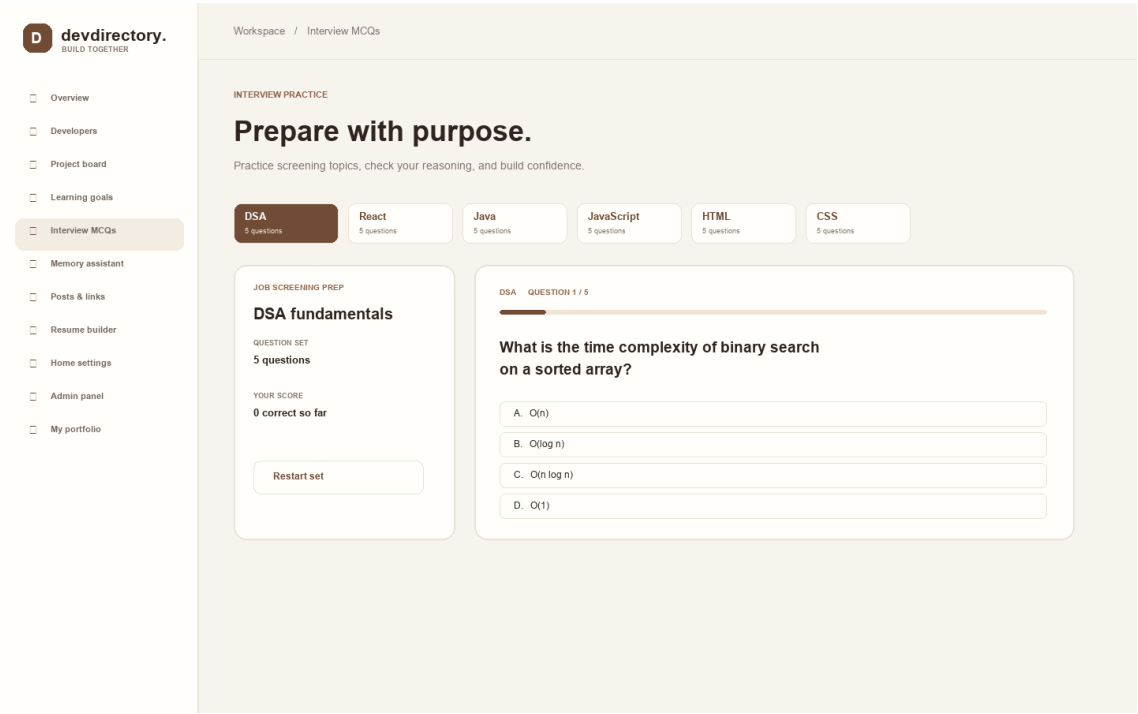
Without Supabase credentials, the app uses local browser storage for supported demo data; that data stays in the current browser. With Supabase configured, authenticated features use the backend policies described in the project schema. GitHub profile/repository lookup uses public GitHub information.

3. Interface previews

Representative screen previews following the warm white and brown design. The memory assistant saves and searches notes locally; interview practice includes six topics, explanations, and personal-best scores.



Memory assistant with searchable local notes



Interview practice with topic selector and question options

4. Run and publish

Run locally

Open the DevDirectory-React-Project folder in VS Code, open a terminal in that folder, then run:

```
npm install
npm run dev
```

Build for hosting

Run `npm run build`. The deployable frontend output is the generated `dist/` folder. For a static Netlify or GitHub Pages deployment, use the Vite build output and configure SPA fallback routing. Supabase-backed features also require production environment variables, database setup, and the Edge Function deployment described in README.md.

Interview practice route

Open `/interview-prep` in the app. Pick DSA, React, Java, JavaScript, HTML, or CSS. Select an answer to see the explanation, finish the 5-question set, then restart or switch topics.

Project notes

The included seeded developers and example content are demonstration data. The repository README documents Supabase setup, environment configuration, and features that require backend credentials.